

STAT 427: Illinois Campus Cluster Usage Analysis

Rushil Bhupta Jiachen Cong Vladislav Fedorov
Rishab Sakalkale Yanlin Wang

1 Introduction

The Illinois Campus Cluster is a campus-wide high-performance computing resource that provides shared computing and storage capacity to researchers across a wide range of disciplines. Users submit computational jobs to the cluster, which are then scheduled and executed according to system resource availability and system policies. During this process, the system automatically generates detailed job and usage logs that record submission, scheduling, execution, and completion information.



Figure 1: Overview of the Illinois Campus Cluster system interface.

Although the Illinois Campus Cluster supports a large and diverse research community, its overall usage patterns and operational behavior have not yet been systematically characterized. In particular, it remains unclear how jobs are submitted over time, how the system responds in terms of waiting time and runtime, and how resource requests vary across workloads. These questions are important because a better understanding of submission behavior, execution schedules, and resource demand can help identify bottlenecks and support more informed scheduling and resource allocation decisions.

The goal of this project is to analyze historical SLURM accounting data from the Illinois Campus Cluster in order to better understand cluster usage from both the user and system perspectives. Our group approaches this broader problem through several complementary lines of analysis, including job submission behavior and execution schedules, user demographics and departmental usage patterns, workload types, and timing patterns of interactive versus batch jobs.

2 Breakdown of Analyses

To address the overall project goal, the analysis is organized into several complementary components, each focusing on a different aspect of cluster usage.

- **Job submission behavior and execution schedules:** This component examines when jobs are submitted, how the system responds in terms of waiting time and runtime, and how resource demand

relates to observed performance patterns. It also investigates GPU jobs as a representative case of resource-constrained workloads.

- **User demographics and departmental usage patterns:** This component studies how cluster usage varies across different user groups and departments, with the goal of identifying disparities in resource utilization.
- **Types of workloads conducted on the cluster:** This component aims to characterize different workload types based on job attributes and resource requests, providing insight into the nature of computational tasks run on the system.
- **Timing patterns of interactive versus batch jobs:** This component analyzes differences in submission timing and execution behavior between interactive and batch workloads.

Each component is presented as a separate section in this report, with consistent emphasis on both exploratory analysis and interpretation. Together, these analyses provide a structured view of cluster usage and help identify key operational patterns and potential bottlenecks.

3 Data Processing

3.1 Data Source Selection

The raw data are organized into multiple folders corresponding to different pipelines and use cases. Since our goal is to study *job submission behavior* and *execution schedules* at the cluster-wide level, we focus exclusively on the production accounting logs under `whole-cluster-usage/`, which represent the full population of jobs.

We do not use the `statclass/` dataset, as it is a small instructional subset and not representative of overall cluster usage. We also exclude auxiliary outputs (e.g., `2025_seff/`), which contain derived efficiency summaries rather than primary scheduling records.

NAME	UPDATED ↓	SIZE
statclass	Feb 5, 2026 by Wyatt Ma...	12 Files
whole-cluster-usage	Feb 5, 2026 by Jedidiah ...	24 Files
slurm_all_jobs_jan15_oct2_2025.zip	Feb 2, 2026 by cheller	184 MB

Figure 2: Overview of the data directory structure.

3.2 Raw Data and Preprocessing

The 2025 accounting data consist of twelve monthly log files with a consistent schema. We concatenate these files into a single dataset covering the full year.

During preprocessing, we apply the following steps:

- Normalize missing values (e.g., `unknown`, `none`, `n/a`)
- Restrict to jobs submitted in 2025
- Simplify job state labels

We also construct key variables used in downstream analysis:

- **Requested memory (MB):** converted from `ReqMem`
- **Waitingtime:** `Start - Submit`
- **Runtime:** `End - Start`

3.3 Job-Level Dataset Construction

The merged logs **may contain multiple records for a single job due to different lifecycle stages** (e.g., pending, running, completed). To avoid double-counting, we construct job-level datasets with exactly one row per `JobIDRaw`.

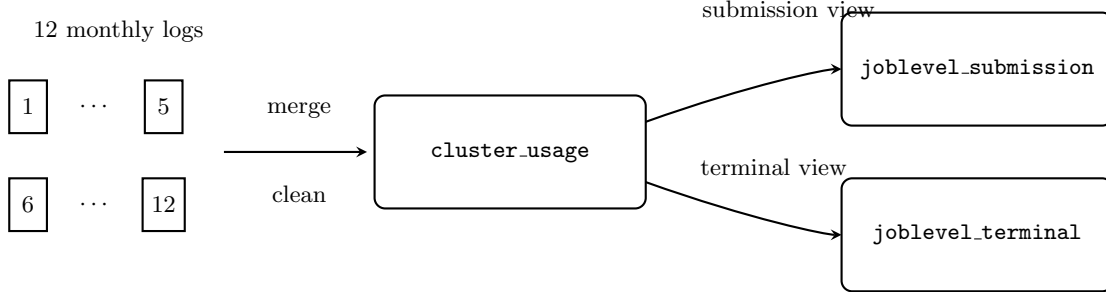


Figure 3: Data construction pipeline from raw logs to job-level datasets.

As shown in Figure 3, the monthly logs are first merged and cleaned to form a unified dataset. We then construct two job-level views:

- **Submission view:** earliest record per job, used for analyzing submission behavior (6,025,326 rows $\times$ 16 columns)
- **Terminal view:** final or most complete record per job, used for analyzing execution outcomes (6,025,326 rows $\times$ 24 columns)

All downstream analyses are conducted using these job-level datasets to ensure consistent interpretation at the job level.

3.4 Final Datasets

The analysis is based on three datasets:

- `master_2025_raw_valid.csv.gz` (cleaned row-level data)
- `master_2025_joblevel_submission.csv.gz` (job-level submission view)
- `master_2025_joblevel_terminal.csv.gz` (job-level execution view)

The two job-level datasets are mainly used in downstream analyses to ensure consistent interpretation at the job level.

4 Job Submission Behavior and Execution Schedules

This section examines operational patterns of job submission and execution on the cluster. We analyze when jobs are submitted, how the system responds in terms of waiting time and runtime, and how resource demand relates to observed performance patterns. We also investigate GPU jobs as a representative case of resource-constrained workloads.

4.1 Submission Timing

We first examine temporal patterns in job submissions using the job-level submission dataset.

As shown in Figure 4, submissions are relatively consistent across weekdays, suggesting stable usage throughout the week. There is a small peak of submission at the start of a week. However, job submissions show a clear hourly pattern, with low activity during early morning hours and peak submissions in the afternoon and late evening. There is a slight drop in submissions during the early evening period (7 to 8 P.M.). The hourly pattern is consistent with students’ daily activity schedules.

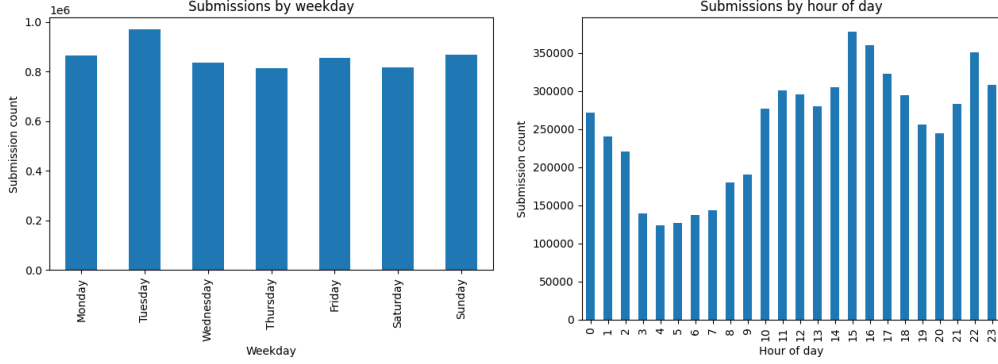


Figure 4: Job submissions by weekday and hour of day.

4.2 System Response: Waiting Time and Runtime

We next examine how the system responds to submitted jobs in terms of waiting time and runtime.

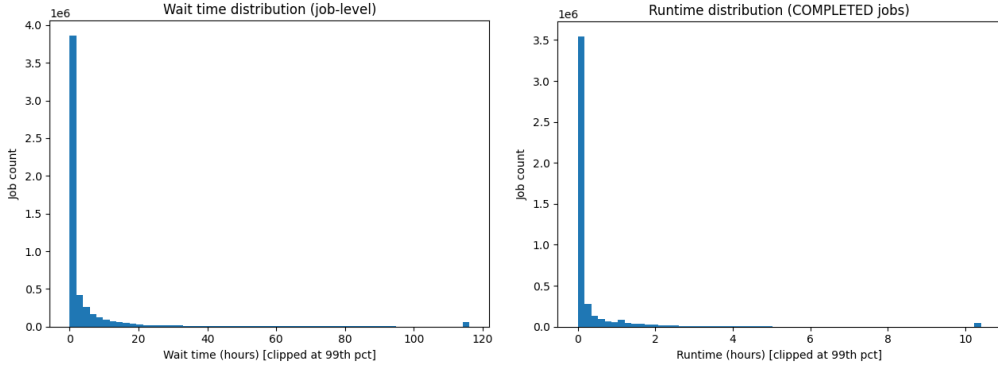


Figure 5: Distribution of waiting time and runtime.

Figure 5 shows that waiting time is highly skewed, with most jobs starting quickly (the median of waiting time is 0.42 hours), but a subset experiencing long delays, about 10% of jobs have waiting time more than 13 hours.

Runtime exhibits a similar right-skewed pattern, where most jobs complete quickly but small amount of jobs run significantly longer. The median of runtime is 0.02 hours; 1% of jobs have runtime more than 10 hours.

4.3 Resource Demand

We analyze requested computational resources across jobs, including CPU, memory, and GPU usage.

As shown in Figure 6, most jobs request relatively small amounts of CPU and memory resources. The median CPU request is 4 CPUs, and the 90th percentile is 16 CPUs. For memory, the median request is about 16 GB, and the 90th percentile is approximately 125 GB.

For GPU jobs, about 89% of jobs request only one GPU, while only around 6% of jobs request five or more GPUs. Although most GPU jobs are small, there are a few very large GPU requests, such as jobs requesting 40 or 64 GPUs.

4.4 GPU Jobs Experience Longer Waiting Times

To investigate whether GPU jobs exhibit different waiting time behavior, we compare GPU and CPU-only jobs.

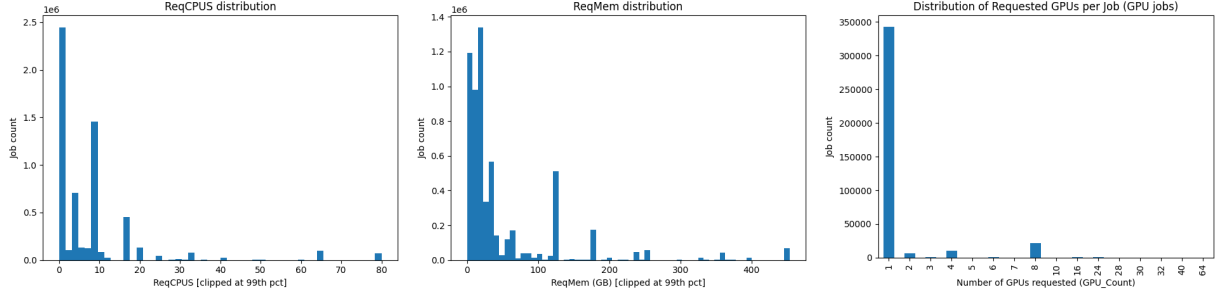


Figure 6: Distribution of requested CPU, memory, and GPU resources.

Because the waiting time data are non-normal and contain extreme values, a non-parametric statistical test was used to compare the two groups. Specifically, the Mann–Whitney U test was applied to determine whether the distributions of waiting times for CPU-only jobs and GPU jobs differ significantly.

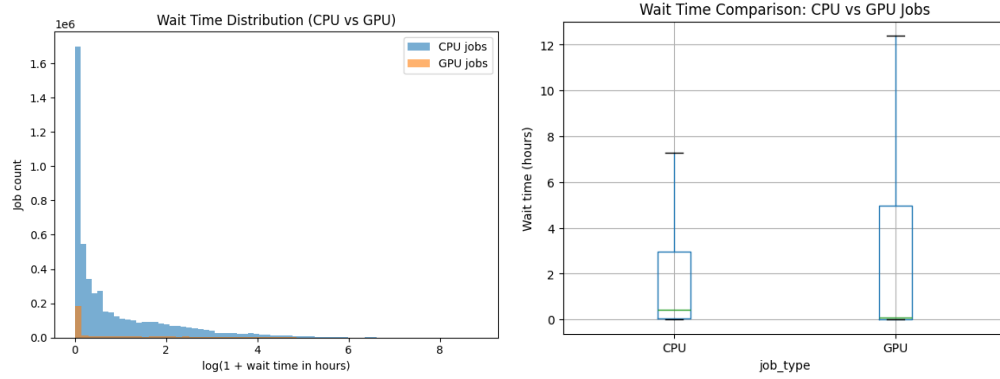


Figure 7: Comparison of waiting times between GPU and CPU-only jobs.

The test returned a p-value of approximately 0 (the Statistic is 1.05×10^{12} , leading to a nearly 0 p-value), which is far below the conventional significance level of 0.05. Therefore, we reject the null hypothesis that the waiting time distributions for CPU and GPU jobs are the same.

The summary statistics and Figure 7 indicate that GPU jobs tend to experience longer waiting times overall. The mean waiting time for GPU jobs is 8.81 hours, compared with 6.83 hours for CPU-only jobs. However, both distributions are highly skewed with long right tails. For example, the median waiting time for GPU jobs is only 0.087 hours, while the median waiting time for CPU-only jobs is 0.44 hours, indicating that many jobs start quickly but a small number experience very long delays.

A likely explanation is that GPU resources are more limited and subject to higher demand, leading to longer queue times for GPU-based workloads.

4.5 Predicting GPU Job Waiting Time

In this part, I implemented a direct regression approach to predict GPU job waiting time. The response variable was defined as the waiting time in hours:

$$\text{WaitTimeHr} = \frac{\text{WaitTimeSec}}{3600}.$$

Before model fitting, I cleaned the data by removing observations with missing, infinite, or negative waiting times. Then I extracted several scheduling-related features, including submission weekday, submission hour, QOS, GPU count, requested CPUs, requested memory, and job priority. To reduce the influence of extreme numerical values, I applied winsorization to the numerical features using the 2.5% and 97.5%

quantiles computed from the training data. Missing numerical variables were imputed by the median and standardized, while categorical variables were imputed by the most frequent value and encoded using one-hot encoding.

Because the waiting-time distribution is highly right-skewed, both models were trained on the log-transformed response:

$$\log(1 + \text{WaitTimeHr}).$$

After prediction, the fitted values were transformed back to the original hour scale using the inverse transformation:

$$\widehat{\text{WaitTimeHr}} = \exp(\hat{y}_{\log}) - 1.$$

I compared two nonlinear regression models: XGBoost and Random Forest. Both models used the same preprocessing pipeline and the same train-test split, so their performance could be compared fairly.

Model	MAE	RMSE	RMSLE
XGBoost	2.5583 hours	24.4489 hours	0.5377
Random Forest	1.9979 hours	21.9759 hours	0.5104

Table 1: Prediction performance of direct regression models without grouping.

Based on these results, the Random Forest model performs slightly better than XGBoost in this direct regression setting. It has lower MAE, RMSE, and RMSLE values, indicating that it produces more accurate predictions on average and also performs better on the log scale.

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(1 + \hat{y}_i) - \log(1 + y_i))^2}.$$

The RMSLE metric is especially useful in this problem because the waiting-time distribution is highly skewed. Compared with RMSE, RMSLE gives more emphasis to relative prediction errors and is less dominated by extremely large waiting times.

The log-scale prediction plot compares the predicted waiting times with the true waiting times. The red dashed line represents the ideal case where the predicted value equals the true value. Points close to this line indicate accurate predictions. From the plot, both models capture the general increasing trend for many observations, especially in the moderate waiting-time range. However, there are still noticeable deviations for very short and very long waiting times, suggesting that predicting extreme waiting times remains difficult.

In addition, the plot shows several horizontal patterns in the predicted values. This is reasonable because both Random Forest and XGBoost are tree-based models, and their predictions are formed from piecewise-constant partitions of the feature space. As a result, multiple observations may receive similar or identical predicted waiting times.

Overall, the direct regression approach provides a simple baseline for wait-time prediction without using a prior classification step. Among the two models, Random Forest gives better predictive performance in this experiment. However, the RMSE is much larger than the MAE for both models, which suggests that a small number of extreme waiting-time observations still lead to large prediction errors. Therefore, it may be useful to compare this direct regression approach with the two-stage classification-plus-regression approach, especially for handling highly imbalanced waiting-time ranges.

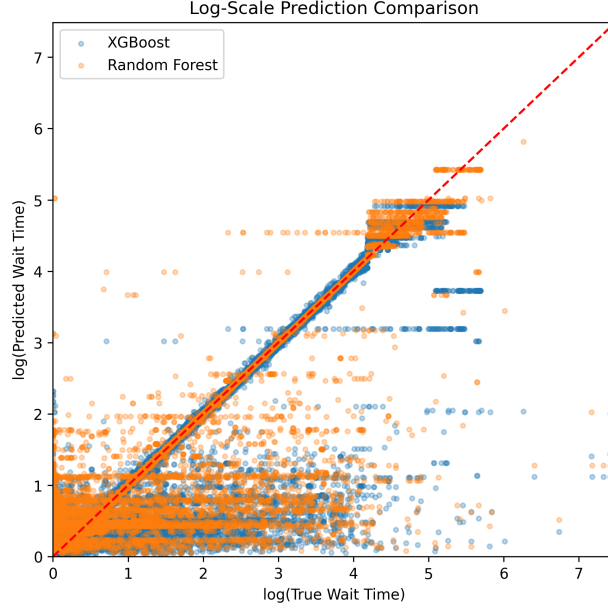


Figure 8: Log-Scale Prediction Comparison (GPU)

4.6 Predicting CPU-only Job Waiting Time

In addition to GPU jobs, I also implemented a separate prediction model for CPU-only jobs. In this analysis, CPU-only jobs were defined as jobs with

$$\text{GPU_Count} = 0.$$

After filtering the data to CPU-only jobs, I applied the same data cleaning and preprocessing procedure as in the GPU-job analysis. Specifically, observations with missing, infinite, or negative waiting times were removed, and the response variable was converted from seconds to hours:

$$\text{WaitTimeHr} = \frac{\text{WaitTimeSec}}{3600}.$$

The predictors used in the model included submission weekday, submission hour, QOS, GPU count, requested CPUs, requested memory, and job priority. Numerical predictors were winsorized using the 2.5% and 97.5% quantiles from the training data to reduce the influence of extreme values. Missing numerical variables were imputed using the median and then standardized, while categorical variables were imputed using the most frequent category and encoded using one-hot encoding.

Because wait time is highly right-skewed, I trained the Random Forest model using the log-transformed response:

$$y_{\log} = \log(1 + \text{WaitTimeHr}).$$

After prediction, the fitted values were transformed back to the original hour scale by

$$\widehat{\text{WaitTimeHr}} = \exp(\hat{y}_{\log}) - 1.$$

The Random Forest model was fitted with 200 trees and maximum tree depth 10. The model performance on the test set is shown below:

Metric	Value
MAE	3.9319 hours
RMSE	23.3144 hours
RMSLE	0.6559

Table 2: Prediction performance of the Random Forest model for CPU-only jobs.

Compared with the GPU-job prediction results, the CPU-only model has a larger MAE and RMSLE. This suggests that CPU-only job waiting times are harder to predict accurately using the current set of features. Although the RMSE is still large, it is close to the GPU-job RMSE, which indicates that both CPU and GPU wait-time prediction are affected by a small number of extreme waiting-time observations.

The log-scale scatter plot compares the predicted waiting times with the true waiting times for CPU-only jobs. The red dashed line represents perfect prediction. Points close to this line indicate accurate predictions. From the plot, the model captures some general increasing trend, but the points are widely dispersed around the diagonal line. There are also several horizontal bands in the predicted values, which is expected for a tree-based model because Random Forest predictions are piecewise constant within terminal nodes.

The plot also shows that the model tends to overpredict some very short waiting times and underpredict some very long waiting times. This pattern indicates that the model has difficulty capturing extreme queueing behavior. Overall, the Random Forest model provides a useful baseline for CPU-only wait-time prediction, but the relatively high MAE and RMSLE suggest that additional features or a more refined modeling strategy may be needed to improve prediction accuracy for CPU jobs.

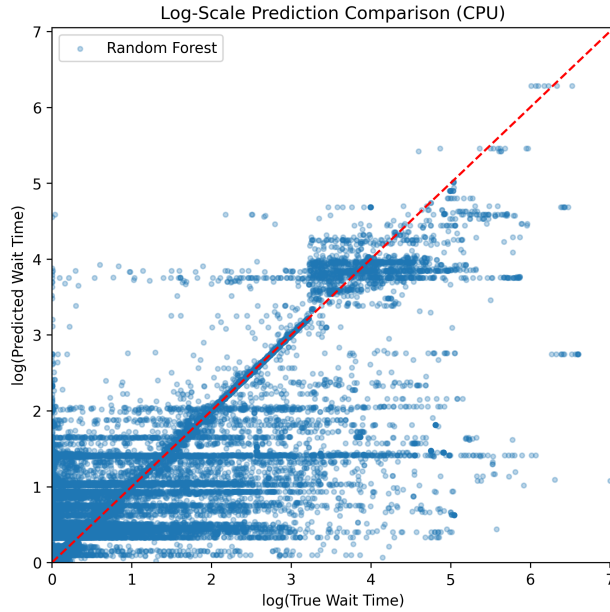


Figure 9: Log-Scale Prediction Comparison (CPU)

4.7 Summary of Findings

Overall, our analysis reveals several key patterns in job submission and execution:

- Job submissions are relatively stable across weekdays, with clear peaks during afternoon and evening hours.
- Waiting time and runtime distributions are highly skewed, indicating occasional long delays.
- Most jobs request modest CPU and memory resources, while GPU usage is concentrated among a smaller subset.
- GPU jobs experience significantly longer waiting times, suggesting resource constraints for GPU workloads.
- The two-stage modeling idea for waiting time prediction is feasible. Multinomial logistic regression model can handle the classification task very well, and linear regression model can predict the exact waiting time quite precisely.

- In addition to the variables available in our dataset, we believe that many other variables could also be useful for waiting time prediction.

These findings provide a foundation for understanding system behavior and motivate further analysis of workload categorization and scheduling dynamics.

5 User Demographics and Departmental Usage Patterns

This section will discuss how cluster usage is distributed among users and departments, with particular emphasis on identifying inequalities in resource usage and potential inefficiencies in access to computational resources.

5.1 User Level Workload Distribution

First, we will discuss how the workload is distributed among individual users. As depicted in Figure 11, it is evident that cluster usage is highly skewed, with a small number of users contributing a large proportion of the total jobs submitted to the cluster.

The top user contributes around 20 percent of the total workload, and the top few users contribute to the total cluster usage. This shows a high heavy-user dependency, implying that a few users heavily depend on the cluster to perform their computations.

This shows that the system usage is not uniformly distributed and may result in conflicts in the future.

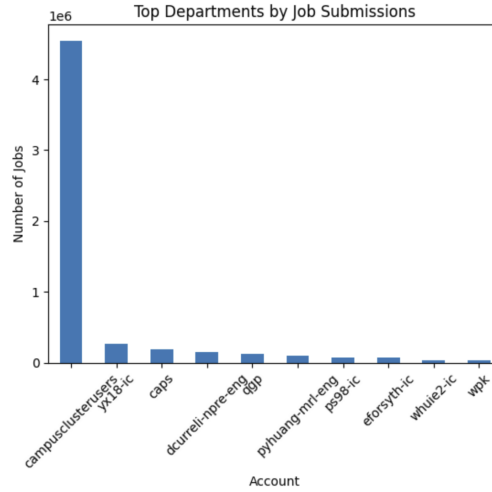


Figure 10: Top Departments By Job Submission

5.2 Departmental Usage Patterns

We now examine the patterns of job submissions at the department level. Figure 12 shows that the pattern of job submissions is heavily skewed toward a small number of departments.

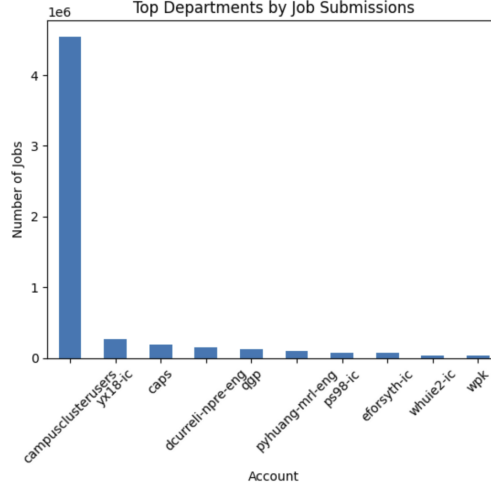


Figure 11: Top Departments By Job Submission

In particular, the campusclusterusers group is heavily responsible for the overall workload. These departments are also responsible for a significant amount of CPU usage, showing that the overall resource usage is heavily centralized.

5.3 CPU Workload Distribution

Further analysis is conducted to understand how CPU workload is distributed among jobs. From Figure 13, it is evident that most jobs require fewer CPU resources, but a small number of jobs require enormous amounts of CPU. This distribution follows a long-tailed distribution, where:

- Most jobs require low computational resources
- A small fraction of jobs require very high computational resources

These jobs may cause bottlenecks for other jobs in the system, leading to delays.

5.4 GPU vs CPU Usage Patterns

We compare CPU and GPU workload distribution to understand resource utilization differences. As shown in Figure 14, GPU jobs account for only about **6% of total jobs**, while CPU jobs dominate the workload.

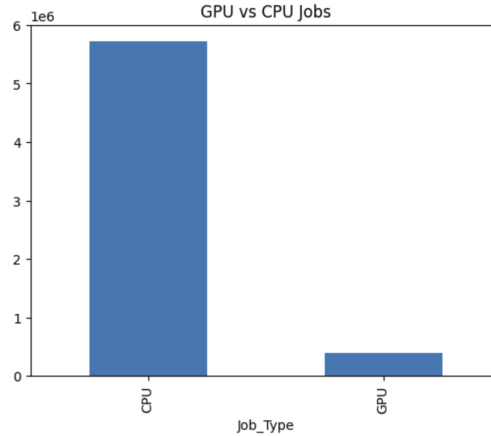


Figure 13: GPU VS CPU Jobs

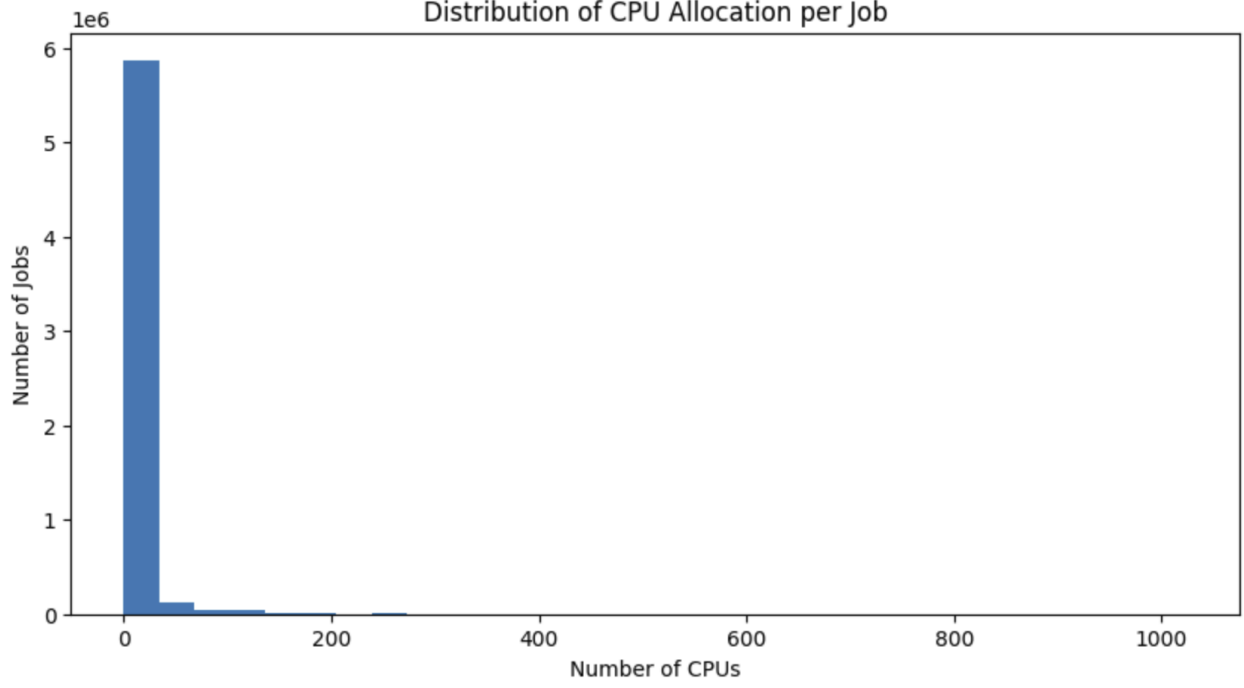


Figure 12: Distribution of CPU Allocation per Job

This suggests that GPU resources are **underutilized overall**, despite their importance for specialized workloads such as machine learning and large-scale simulations.

The imbalance indicates potential opportunities to **increase GPU adoption** or improve workload allocation strategies.

5.5 GPU Usage Inequality Across Users

Finally, we analyze how GPU usage is distributed across users. As illustrated in *Figure 15*, GPU usage is **extremely concentrated among a small group of users**.

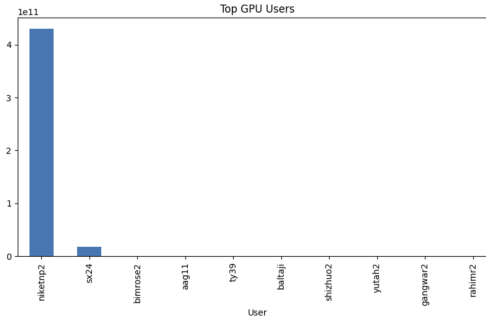


Figure 14: Top GPU Users

The majority of users have little to no access to GPU resources, while the **top 10% of users account for nearly all GPU usage**.

This highlights a significant **resource access inequality**, suggesting that GPUs are primarily used by specialized users or research groups.

Such imbalance raises important questions about fairness and accessibility in resource allocation.

	job_count	job_pct	row_count	row_pct
outcome				
Cancelled	639041	10.59	641112	10.50
Completed	4644274	76.98	4662342	76.36
Failed	585157	9.70	586554	9.61
Node Failed	218	0.00	220	0.00
Other	845	0.01	850	0.01
Out of Memory	43313	0.72	43546	0.71
Requeued	9	0.00	9	0.00
Running	301	0.00	3600	0.06
Stuck PENDING	3791	0.06	45464	0.74
Timeout	116144	1.93	122413	2.00
Total unique jobs: 6033093				
Total rows: 6106110				

Figure 15: Job State Counts

5.6 Summary of Findings

From the analysis above, it is evident that there are significant patterns in the data:

- User workload distribution is heavily skewed in favor of heavy users in terms of cluster usage
- Departmental usage is heavily centralized in favor of a few departments in terms of usage
- CPU workloads exhibit a long-tail distribution in favor of a few users in terms of jobs submitted
- GPU usage is characterized by
 - Low usage in general (6 percent)
 - High concentration in favor of a small set of users

From these results, it is apparent that there is a need to improve resource allocation, GPU access, and to monitor heavy user usage.

6 Workload Types on the Cluster

6.1 Introduction

One key variable stored for each job in the SLURM data is "JOB.STATE". This variable stuck out as particularly interesting for its potential to track jobs: by examining the evolution of job states individually (tracking a particular job's state over time) and cluster-wide (tracking failure rates over time) we could discover correlations between other job characteristics or extreme cluster conditions that lead to job failure.

6.2 Data Cleaning + EDA

Below is the counts and rates of each job state, and cluster-wide counts of these states over time:

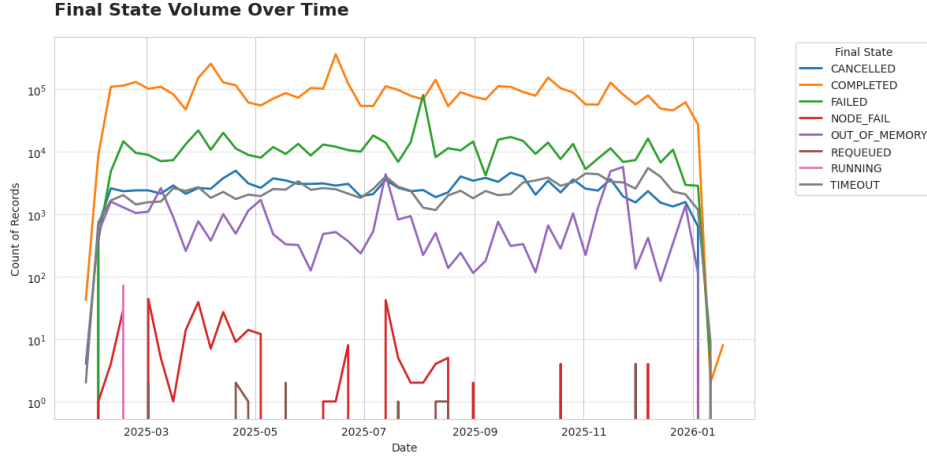


Figure 16: Job State Counts Over Time

We can note that:

- 76% of jobs are completed
- 43k jobs are out-of-memory
- 20% of jobs are either cancelled or failed

Note that in the original data, there were several thousand unique job states following the format "CANCELLED by #####", with the number representing the Job ID of a different job. While these job states are simply represented as CANCELLED in the plots above, additional analysis into an example of such a state was performed to understand the impact of these jobs - are they one-off jobs run by a user to quickly kill off jobs or part of an unusual/catastrophic/unprovoked event?

One of the largest-impact examples is the job with ID 1147665. This job killed 21586 jobs with names like:

- ddp-h200-torch
- ddp-l40s-torch
- aims_light_model_test
- full_ae-h200

Importantly, all these jobs killed by job 1147665 were created by one user. Similar analysis on other, large-impact job-killers reveals a similar dynamic: often times, users submit jobs that kill a large amount of their own jobs. Because these are intentional, controlled job cancellations, we are justified in labeling all "CANCELLED by #####" jobs as "CANCELLED" jobs. Further confirmation from our meeting with our client supports this conclusion.

6.3 Exploring Potential Areas of Analysis

Now, as mentioned in the introduction, one of the initial motives in analyzing job states was to track a job's state over time. Below is a list of the average number of rows per unique job organized by their final job state.

We notice that:

- 1 unique job = 1 row in the SLURM dataset
- Only pending and running states have multiple rows per job

outcome	count	mean	std	min	25%	50%	75%	max
Cancelled	639041.0	1.003241	0.065675	1.0	1.0	1.0	1.0	10.0
Completed	4644274.0	1.003890	0.062251	1.0	1.0	1.0	1.0	2.0
Failed	585157.0	1.002387	0.048943	1.0	1.0	1.0	1.0	3.0
Node Failed	218.0	1.009174	0.095562	1.0	1.0	1.0	1.0	2.0
Other	845.0	1.005917	0.076741	1.0	1.0	1.0	1.0	2.0
Out of Memory	43313.0	1.005379	0.073148	1.0	1.0	1.0	1.0	2.0
Requeued	9.0	1.000000	0.000000	1.0	1.0	1.0	1.0	1.0
Running	301.0	11.960133	0.267841	10.0	12.0	12.0	12.0	12.0
Stuck PENDING	3791.0	11.992614	0.112296	7.0	12.0	12.0	12.0	12.0
Timeout	116144.0	1.053976	0.237671	1.0	1.0	1.0	1.0	3.0

Figure 17: Job State Counts

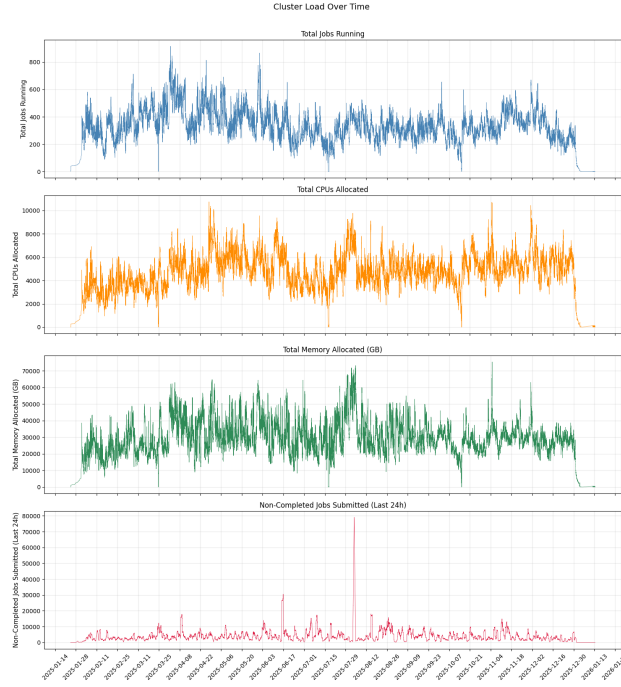


Figure 18: Job State Counts

This unfortunately indicates that job states are terminal: we are unable, based on the current data, able to track the evolution of jobs.

This also indicates to us that the PENDING job state is even more uncommon than previously perceived, since most jobs get stuck in PENDING, and artificially add rows into the SLURM data - something that doesn't happen in other job states. As further confirmation, we notice that:

- 88.7% of PENDING jobs only request 1 CPU
- 70% of jobs requested by 1 user in the “astro-physics” group

Since the PENDING jobs come from a single user, we can remove them from future analyses. Now, finally, we look at cluster-wide job states over time. Below is the plot of requested resources and incomplete jobs over time:

From the plots above, we can see a major spike in incomplete events in late July, which seems to coincide with a sizable increase in requested resources. It would be interesting to analyze if a large increase in requested/allocated resources leads to failure in jobs.

6.4 Next Steps

Based on the previous analyses, there are a few interesting questions/topics to explore:

- Can we predict, based on cluster-wide statistics and job requests, whether a job will not complete?
 - I.e. is there a correlation between ReqMem and total memory used across the cluster and node failure / cancelled jobs?
- Do certain users/accounts/partitions have high failure rates?
 - Could indicate users who need guidance on job configuration

7 Analysis of Cluster Usage and Reservation Times

7.1 Introduction

This analysis investigates how requested runtimes compare to observed execution times across a large-scale cluster workload. The goal is to quantify the extent of overestimation, identify patterns across workloads, and estimate the resulting inefficiency in reserved compute resources.

7.2 Data and Preprocessing

The dataset consists of approximately 6.1 million cluster job records, including job metadata, timestamps, and resource requests. To improve data quality and reduce noise, the following preprocessing steps were applied:

- Removed columns with predominantly missing or uninformative values, defined as:
 - More than 6 million missing entries
 - More than 6 million entries equal to 0, "Unknown", or "00:00:00"
- Filtered out invalid jobs:
 - Jobs with zero runtime (`ElapsedRaw` = 0)
 - Jobs with invalid or missing end timestamps
- Converted relevant fields to numeric types (e.g., runtime, time limits)

The final dataset contains cleaned job-level records with valid runtime and resource information.

7.3 Methodology

To compare requested and actual runtimes, all time units were standardized:

- Actual runtime (`ElapsedRaw`) is recorded in seconds
- Requested runtime (`TimeLimitRaw`) is recorded in minutes and converted to seconds

Runtime utilization is defined as:

$$\text{Utilization} = \frac{\text{Actual Runtime}}{\text{Requested Runtime}}$$

Unused compute was estimated as:

$$\text{Unused Seconds} = \text{Requested Runtime} - \text{Actual Runtime}$$

$$\text{Unused CPU-hours} = \frac{\text{Unused Seconds} \times \text{Requested CPUs}}{3600}$$

Runtime utilization was also analyzed across cluster partitions to identify structural differences between high-volume and specialized workloads.

To estimate potential efficiency improvements, a counterfactual simulation was performed:

- For each partition, the 90th percentile of observed runtime was computed
- Requested runtimes were capped at this value
- Resulting unused CPU-hours were recomputed

7.4 Results

The analysis reveals strong and consistent overestimation of requested runtimes:

- Median job runtime: 1.2 minutes
- Median requested runtime: 240 minutes
- Median utilization: 0.57%
- 95.5% of jobs use less than half of their requested time
- 800 million unused CPU-hours

Runtime utilization varies significantly across partitions:

- High-volume partitions (e.g., **secondary**, **IllinoisComputes**) exhibit extremely low utilization
- Lower-volume research partitions show much higher utilization (up to ~60%)

Runtime overestimation is especially shown among short jobs:

- Median requested runtime is approximately 175x actual runtime
- For longer jobs:
 - >5 minutes: 11.1x
 - >10 minutes: 7.5x
 - >30 minutes: 3.8x

Among jobs under 2 hours:

- 95.5% request at least 2x their runtime
- 90.4% request at least 4x
- 84.0% request at least 10x

Capping requested runtimes at the 90th percentile of observed runtime per partition results in a substantial reduction in unused compute.

7.5 Discussion

The findings indicate that runtime requests are used as conservative upper bounds rather than predictive estimates, likely driven by user incentives to avoid job termination.

While scheduling systems may mitigate some inefficiency through backfilling, the magnitude of overestimation suggests that meaningful gains are still possible.

Potential improvements include:

- Providing users with historical runtime feedback
- Recommending runtime estimates based on similar jobs
- Adjusting default limits for high-volume partitions
- Potentially implementing an automatic re-request for faster jobs

8 Future Work

Building on the current analysis, the next stage of the project will extend each component of the study in a structured way.

- **Job submission behavior and execution schedules:** We plan to further investigate the relationship between submission timing and workload characteristics. In particular, we will refine the definitions of some important thresholds for classification models in Stage 1; and use more advanced models, such as random forest models, neural network models, to improve the precision of prediction in Stage 2. Then, we will build a prediction model for CPU-only jobs with similar idea. We will also explore whether submission patterns are influenced by external academic factors, such as conference or journal deadlines.
- **User demographics and departmental usage patterns:** Moreover, the analysis has highlighted considerable disparity in the utilization of resources within the cluster. A small number of users are contributing to a disproportionately large number of total job submissions, while a considerable workload is also centralized within a limited number of departments. This has highlighted the centralized usage of resources within the cluster. CPU resources are widely utilized within the jobs, while the utilization of CPU resources is not uniform, highlighting a considerable disparity in the distribution of resources. A small number of jobs are utilizing a considerable percentage of CPU resources. On the other hand, GPU resources are largely underutilized, while a considerable number of resources are centralized within a limited number of users. Based on the above analysis, future work will be conducted to further improve the accessibility of GPU resources and balance the allocation of resources among users and departments.
- **Workload types on the cluster:** Analyzing job states reveals a few things. First, job states are terminal: the data contains the final snapshot of jobs, not a time series tracking the evolution of a job through its lifetime. Additionally, the job states contain many "CANCELLED by #####" jobs, which upon further analysis and confirmation from our clients, are not indications of system failure, but the result of user behavior where a user runs a job that terminates other jobs. Similarly, PENDING jobs have an artificially-inflated presence in the data due to the actions of a single user. Because of this, the "CANCELLED by #####" are relabeled as "CANCELLED" and PENDING jobs are removed from future analyses. Finally, plotting requested resources and the number of incomplete jobs over time revealed a massive spike in such jobs in July 2025 that coincided with an increase in requested resources. This indicates the possibility in a relationship between requested/allocated resources and job failures. This is the future step of job state analysis, but might be de-prioritized in case more work/man-hours is required for work on the resource utilization analysis.
- **Analysis of Cluster Usage and Reservation Times:** This analysis shows that runtime overestimation is widespread and largely driven by user behavior rather than system defaults. Because CPU

resources are strictly reserved and unused capacity cannot be reallocated, this overestimation directly reduces cluster throughput by limiting how many jobs can run concurrently. Given these findings, future work could focus on better understanding how runtime overestimation affects queue wait times and overall system availability. Potential directions include exploring simple runtime guidance or feedback mechanisms to help users make more accurate requests. However, even without further modeling, the current results already highlight a clear opportunity to improve cluster efficiency through better runtime calibration.

Overall, these extensions aim to provide a more structured and interpretable understanding of cluster usage, and to support more informed decision-making regarding scheduling and resource allocation.